Cybersecurity Lab: SIEM-Based Detection Using Wazuh

Prepared by:
KIROLOS SHEHATA

Organization:
INSTITUTE OF APPLIED ENGINEERING

Date:
03-17-26

Course/Program:
CYBER LAB INTERNSHIP

Executive Summary

This project demonstrates a simulated enterprise cybersecurity environment using the Wazuh SIEM platform to detect and analyze malicious activity. The lab was built using Proxmox virtualization and consists of a red team attacker machine (Kali Linux), two victim systems (Ubuntu and a Windows domain client), and a blue team SIEM server (Wazuh). Multiple attack scenarios were conducted, including network scanning (Nmap), SSH brute-force attacks, file tampering, and Active Directory enumeration with Kerberoasting techniques. These activities generated logs on the victim systems that had agents deployed on them, which were collected and analyzed by Wazuh. Wazuh successfully detected and generated alerts for each attack scenario by applying built-in and custom detection rules that were written. The platform also provided insight into system behavior, detection capabilities, and potential improvements in security configurations. This documentation outlines the lab architecture, Wazuh SIEM architecture, detection mechanisms, and detailed attack simulations. It also discusses challenges encountered, lessons learned, and recommendations for improving the lab and detection capabilities.

LAB ARCHITECTURE

Physical Layer

- **3 Physical HP Servers**
- **Proxmox installed on each**

Virtual Layer

- Server 1 → Blue Team (Wazuh and Domain Controller)
- Server 2 → Red Team (Kali)
- Server 3 → Victims (Ubuntu + Windows DC)

Network

VLAN-based segmentation:

- VLAN 20 → Blue
- VLAN 30 → Red
- VLAN 40 → Victim

The VLAN's allow proper segmentation of network so traffic is not all running through one network and helps enforce firewall rules better through OPNsense.

Routing

OPNsense firewall handles traffic between VLANs. It can block and allow certain connection from the different VLANs. For the sake of the lab and demonstration, traffic is allowed between VLAN 30 and 40 to simulate real-world scenarios where an attacker has network access to target systems and can initiate reconnaissance and attack activities.

WAZUH ARCHITECTURE

---

1. Wazuh Agent

- The Wazuh agents are deployed directly on the endpoints, for the lab, there are 2 that will be installed, one on the Ubuntu and the other on the Windows. They can be deployed through the Wazuh dashboard by choosing which OS and version and the script will be generated.

The Wazuh agents collects logs like

- Authentication logs
- File changes
- System activity
- Windows Event Logs

Wazuh Agent

---

2. Wazuh Manager

- Central processing system
- Applies detection rules
- Correlates events

Wazuh Manager (Server)

---

3. Wazuh Indexer

- Stores logs (Elasticsearch/OpenSearch)
- Allows searching and inexing

Indexer (OpenSearch database)

---

4. Wazuh Dashboard

- Web interface
- Displays alerts
- Allows searching and threat hunting

Dashboard (Web interface)

Detection Mechanisms

1. Rule-Based Detection

- Uses XML rules
- Matches patterns in logs

Example:

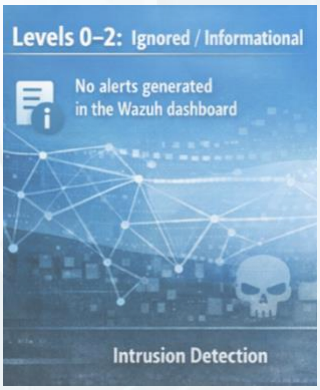
```
<rule id= "2502" level = "10">
  <match>more authentication failures;|REPEATED login failures</match>
  <description>syslog: User missed the password more then on time>/description>
```

- The rule id gives the rule a unique identification method to identify that specific rule and is not standardized unlike MITRE ATT&CK ID (T1110).
- The Match looks for literal text and matches anywhere in the log line; multiple match tags must all match.
- `<if_sid>` matches the rule to an existing one in Wazuh, might be default.
- The description is the readable explanation to what the alert is, any text can be added.
- Groups can also be made, and group policies can be written to appy to multiple agents at one time, better for enterprise situations.

---

2. File Integrity Monitoring (FIM)

- Tracks file changes
- Detects:
  - File creation
  - Modification
  - Deletion



3. Vulnerability Detection

- Compares installed software with CVE databases

4. Compliance Monitoring (SCA)

- Compares system config to:
  - *CIS*
  - *NIST*
  - *PCI-DSS*

5. Active Response

- Can automatically:
  - Block IPs
  - Disable users
  - Modify firewall rules

White_list: Does not block any IP addresses in that list

Disable-account: Blocks that IP usually brute force attempts, or compromised behavior

Restart-wazuh: Restarts the Wazuh agent on the machine where the log comes in from

Firewall-drop: Drops all traffic from that IP, on the network level, does not get network access.

Host-deny: Blocks IP by getting added to a deny list, (/etc/hosts.deny) but generally less reliable than firewall-based blocking.

Route-null: Automatically blocks malicious IP addresses by adding them to a blackhole routing table.

Win_route_null: The windows version and does the same thing.

Active responses must be configured so it points at a rule, whenever the alert comes in and the rule is triggered, the active response activates.

```
<!-- Active response -->
<global>
  <white_list>127.0.0.1</white_list>
  <white_list>^localhost.localdomain$</white_l
  <white_list>10.0.2.3</white_list>
</global>

<command>
  <name>disable-account</name>
  <executable>disable-account</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>

<command>
  <name>restart-wazuh</name>
  <executable>restart-wazuh</executable>
</command>

<command>
  <name>firewall-drop</name>
  <executable>firewall-drop</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>

<command>
  <name>host-deny</name>
  <executable>host-deny</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>

<command>
  <name>route-null</name>
  <executable>route-null</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>

<command>
  <name>win_route-null</name>
  <executable>route-null.exe</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>
```

Example Active Response command:

```
<active-response>
<command>firewall-drop</command>
<location>local</location>
<rules_id>5710</rules_id>
<timeout>600</timeout>
</active-response>
```

Red Team Simulation

Attack Scenarios

1. Nmap Port Scanning

Command:

```
nmap -sS -T4 <target>
```

What happened:

- Generated multiple connection attempts
- Logged by UFW:

```
UFW BLOCK SRC=... DPT=...
```

Detection:

- Custom rule matched `UFW BLOCK`
- Alert triggered, and shown on dashboard

timestamp	agent.name	rule.description	rule.level	rule.id
Mar 12, 2026 @ 13:14:55.7...	Ubuntu	Possible port scanning activity detected	10	100300
Mar 12, 2026 @ 13:14:55.7...	Ubuntu	Possible port scanning activity detected	10	100300

```
<rule id="100300" level="10">
 <match>UFW BLOCK</match>
 <description>Possible port scanning activity detected</description>
 <group>reconnaissance,network</group>
</rule>
```

2. SSH Brute Force

Command:

```
ssh fakeuser@target
```

Logs generated:

```
Failed password
Invalid user
```

Detection:

- Wazuh rule matched authentication failures
- Brute force alert triggered

Points to rule 5710, and that is where it matches, and prints. If you refer to the active response, you can see that this rule is tied to a firewall drop AR.

Frequency means 8 failed login attempts, within (timeframe)120 seconds from the same source IP, which can be seen on the next page, when inspecting the full document.

---

timestamp	agent.name	rule.description	rule.level	rule.id
Mar 26, 2026 @ 11:14:55.7...	Ubuntu	syslog: User missed the password more than one time	10	2502
Mar 26, 2026 @ 11:14:53.7...	Ubuntu	sshd: brute force trying to get access to the system. Non existent user.	10	5712

```
┌──(kali㉿Kali-Red)-[~]
└─$ ssh ubuntu@10.0.40.7
ubuntu@10.0.40.7's password:
Permission denied, please try again.
ubuntu@10.0.40.7's password:
Permission denied, please try again.
ubuntu@10.0.40.7's password:
ubuntu@10.0.40.7: Permission denied (publickey,password).
```

```
<rule id="5712" level="10" frequency="8" timeframe="120" ignore="60">
  <if_matched_sid>5710</if_matched_sid>
  <same_source_ip />
  <description>sshd: brute force trying to get access to the system. Non existent user.</description>
  <mitre>
    <id>T1110</id>
```

Inspecting Document Details

The document is an expanded more detailed view of the alert details

1. Agent Information:

Agent.id: The unique ID of the monitored endpoint

Agent.name: the name of the host machine generating the loh, (Ubuntu)

Agent. Ip: the IP address of the monitored system.

This information relates to the machine that the alert is against

---

2. Source of the Attack:

Data.scrip: (10.0.30.131) The IP address attempting the login

Data.src**user: ubuntu** ⇒ The username used in the attempt

---

3. Log and Decoder Info

Decoder.name: sshd ⇒ The log was parsed as an SSH daemon event

Full_log: Shows the username does not exist on the system, there is a failed login attempt, the connection came from a specific IP and port

Document Details

View surrounding documents | View single document

Table | JSON

t _index	wazuh-alerts-4.x-2026.03.26
t agent.id	001
t agent.ip	10.0.40.7
t agent.name	Ubuntu
t data.srcip	10.0.30.131
t data.srcuser	ubuntu
t decoder.name	sshd
t decoder.parent	sshd
t full_log	Mar 26 15:14:52 ubuntu-blue-s3-Standard-PC-i440FX-PIIX-1996 sshd[4812]: Failed password for invalid user ubuntu from 10.0.30.131 port 38654 ssh2
t id	1774538093.11675
t input.type	log
t location	journald
t manager.name	wazuh-server
t predecoder.hostname	ubuntu-blue-s3-Standard-PC-i440FX-PIIX-1996
t predecoder.program_name	sshd
t predecoder.timestamp	Mar 26 15:14:52

3. File Tampering

Command:

```
sudo touch /etc/hackedfile
sudo rm /etc/hackedfile
```

Detection:

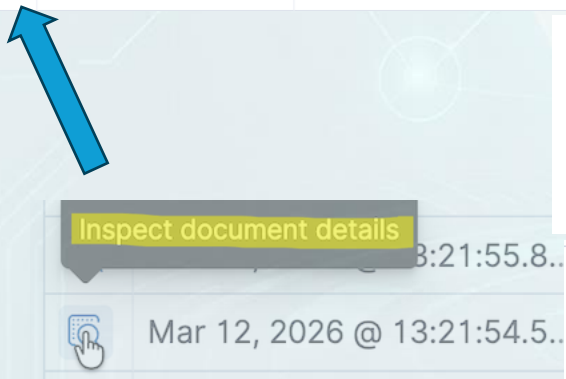
- File Integrity Monitoring triggered:
  - File added
  - File modified
  - File deleted

Mar 12, 2026 @ 13:21:54.5...	Ubuntu	File added to the system.	5	554
Mar 12, 2026 @ 13:21:54.5...	Ubuntu	File added to the system.	5	554

t full_log	File '/etc/wazuh-demo-file' added Mode: realtime

- A new file was created within a monitored directory. While file creation is a common event, it becomes significant when occurring in sensitive or restricted paths, or when the source of the file is unknown.
- Once again, inspecting the full log will give us more details we need, like what file specifically.

Mar 12, 2026 @ 13:22:43.0...	Ubuntu	Integrity checksum changed.	7	550
Mar 12, 2026 @ 13:22:43.0...	Ubuntu	Integrity checksum changed.	7	550



t full_log	File '/etc/wazuh-demo-file' modified Mode: realtime Changed attributes: size,mtime,md5,sha1,sha256 Size changed from '0' to '6' Old modification time was: '1773336113', now it is '1773336161' Old md5sum was: 'd41d8cd98f00b204e9800998ecf8427e'
t full_log	New md5sum is : '8feb211fef744a43469cf07400094c70' Old sha1sum was: 'da39a3ee5e6b4b0d3255bfef95601890afd80709' New sha1sum is : '774be3106edbb6d80be36dbb548d62401dcfa0fe' Old sha256sum was: 'e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855' New sha256sum is : 'dc122cd797e76d1e0b07efe6262829098581816f1727d9a883bd4052a4e659ef'

Shortly after creation, the file's checksum (hash) changed, indicating that its contents were modified. This behavior suggests that the file may have been altered post-creation, potentially to introduce or modify executable code, evade detection, or update a payload.

From the Wazuh log details:

- The file `/etc/wazuh-demo-file` was modified in **real-time**
- Multiple attributes changed, including:
  - **Size** (from 0 to 6 bytes)
  - **Modification time (mtime)**
  - **Checksums (MD5, SHA1, SHA256)**

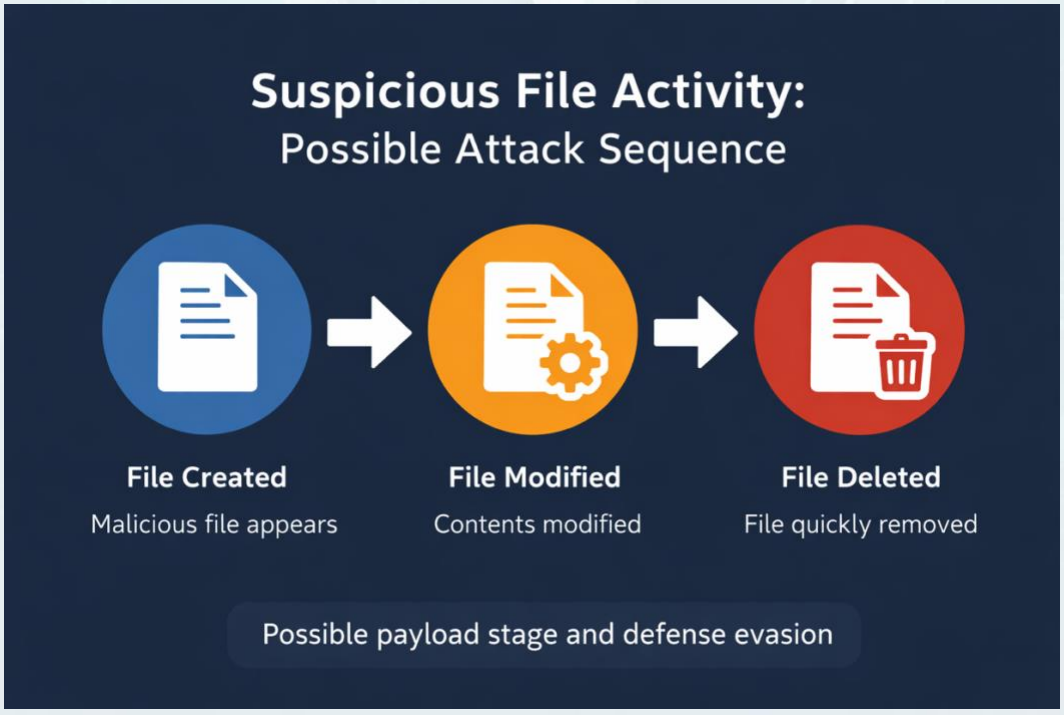
The change in cryptographic hash values (MD5, SHA1, SHA256) is very significant. These hashes uniquely represent the file's contents, meaning any alteration, even a single byte, results in a completely different checksum.

Mar 12, 2026 @ 13:22:48.5...	Ubuntu	File deleted.	7	553
Mar 12, 2026 @ 13:22:48.5...	Ubuntu	File deleted.	7	553

```
t  full_log        File '/etc/wazuh-demo-file' deleted
                   Mode: realtime
```

- The file was subsequently deleted. This action may indicate an attempt to remove evidence of the file's existence and activity, which is a common tactic used by attackers to avoid forensic detection.

- The file deletion event further strengthens the assessment of suspicious activity. When correlated with the preceding file creation and integrity modification alerts, the deletion suggests intentional removal of artifacts, a behavior commonly associated with anti-forensic techniques and short-lived payload execution.



ACTIVE DIRECTORY ENUMERATION + KERBEROASTING

Active Directory enumeration was performed to identify domain users, groups, and service accounts within the environment. Using commands such as `net user /domain` and tools like Impacket, specifically `GetUserSPNs.py`, I was able to enumerate accounts associated with Service Principal Names (SPNs), which are potential targets for Kerberoasting.

Once these accounts were identified, a Kerberoasting attack was executed by requesting Kerberos service tickets (TGS) for those SPNs using Impacket. These tickets are encrypted using the service account's password hash, typically with RC4 encryption, making them susceptible to offline password cracking.

The extracted ticket hashes were then saved and can be cracked offline using tools such as Hashcat or John the Ripper, allowing an attacker to potentially recover plaintext credentials without further interaction with the domain controller.

This activity generated Windows Security Event ID 4769, indicating that a Kerberos service ticket was requested. The Wazuh agent collected these logs, and custom detection rules were used to identify suspicious characteristics such as RC4 encryption and abnormal ticket behavior.

This demonstrates how attackers move from reconnaissance (enumeration) to credential access (Kerberoasting and password cracking), and how SIEM platforms like Wazuh can detect these activities through log collection, event correlation, and rule-based analysis

Alert Analysis

- Timestamp
- Source IP
- Target machine
- Rule triggered
- Severity level

Inspect document details

	agent.name	rule.description	rule.level	rule.id
Mar 12, 2026 @ 13:35:06.4...	Win-DC01	Possible Keberoasting attack	12	110002
Mar 12, 2026 @ 13:34:09.6...	Win-DC01	Possible Keberoasting attack	12	110002

```
t  data.win.system.message        "A Kerberos service ticket was requested.

                                  Account Information:
                                          Account Name:          student1@LAB.LOCAL
                                          Account Domain:        LAB.LOCAL
                                          Logon GUID:            {6c790706-4e46-d89f-
                                  fb61-dad918238693}
```

- In this case, we can see a rule labeled “Possible Kerberoasting attack” with a high severity level of 12, indicating suspicious activity related to Kerberos service ticket requests.

- The alert is based on a Windows Security Event, that the deployed Wazuh agent retrieved, specifically Event ID 4769, which indicates that a Kerberos service ticket was requested.

- In the log details we can see that a service ticket was requested for the account student1@LAB.LOCAL, which confirms that Kerberos activity was successfully generated and logged.

- The rule looks for the specific RC4 encryption type. It is important to note that Kerberos service ticket requests, such as Event ID 47769, are normal in a windows domain environment. However, this alert is triggered based on specific characteristics, like encryption type, and ticket options, which can indicate suspicious behavior associated with Kerberoasting attacks.

- This highlights the importance of cybersecurity awareness and contextual analysis, where individual events are not viewed in isolation but are correlated with other activities to determine whether behavior is normal or potentially malicious

Challenges Faced

During the development of this lab, several technical challenges were encountered that impacted system functionality and detection accuracy. This required troubleshooting and time to resolve.

- **One issue involved Kerberos authentication. Kerberos is highly sensitive to time synchronization, and even small discreptencies between systems, caused authentication failures and prevented ticket generation and *Impacket* execution.**
- **Another issue was DNS Misconfiguration. The domain controller could not be properly resolved, which caused errors like "domain does not exist". This required correction for Active Directory to communicate properly.**
- **I also encountered an issue with alerting on Kerberos ticket generation. Kerberos ticket generation required specific service requests for it to generate a log alert. For example, it required a new generation of a 4768 (AS) and 4769 (TGS) event. Also, if there were any old valid cached tickets, a new ticket would not be requested, therefore no alert would be generated.**
- **I also encountered some issues with VLAN communication due to firewall rules and segmentation. While segmentation is important for security, the configuration initially blocked necessary traffic between the attacker and victim machines and ultimately prevented attack simulations from working properly.**
- **Resource limitations were another significant challenge. Due to limited memory on the physical servers, I was unable to fully take advantage of the servers and run multiple virtual machines simultaneity, without the Wazuh manager crashing. This just required careful allocation of resources, optimization, and prioritization of critical systems (like OPNsense, Wazuh SIEM) to maintain stability and performance.**

These challenges highlight the importance of proper system configuration, synchronization, as well as troubleshooting and resource planning in real-world cybersecurity environments. Misconfigurations in any of these can disrupt critical services such as Active Directory and Kerberos authentication, prevent log generation, and impact the effectiveness of SIEM-based detection systems

CONCLUSION

- Successfully built a real-world SOC environment using Wazuh
- Demonstrated real-world attack and detection scenarios (scanning, brute force, Kerberoasting, file tampering)
- Explored IT hygiene and system monitoring capabilities.
- Utilized threat hunting and log analysis features
- Implemented vulnerability scanning and compliance monitoring.
- Examined malware detection and file integrity monitoring
- Configured custom rules and active response mechanisms

This project demonstrates how attackers generate activity and how defenders rely on logs, rules, and analysis to detect, investigate and respond to threats effectively and efficiently. Future improvement would involve utilizing the malware detection and vulnerability scanning, in alignment with NIST security controls, to generate automated alerts for a system to be classified as vulnerable, and how to remediate, as well as detecting malware on systems, or extensions that may have malware, and display an alert about it.